SPEED DETECTOR USING ARDUINO AND IR SENSORS

BY:

A.Sharan (23R11A05U8)

D.Romith (23R11A05V4)

K.Advaith (23R11A05W7)

INTRODUCTION

- With the rapid growth in the number of vehicles, monitoring vehicle speed has become essential for ensuring road safety. Overspeeding is one of the major causes of accidents, leading to injuries and loss of life.
- Traditional speed detection systems such as radar guns and camera-based systems are expensive and not suitable for all environments. Therefore, there is a need for a simple, low-cost, and efficient system.
- This project presents a **Speed Detector System using Arduino and IR sensors**, which measures the speed of a moving vehicle using time and distance principles.

OBJECTIVES

The main objectives of this project are:

- To design a system that can measure the speed of a moving vehicle.
- To use **IR sensors** for detecting vehicle movement.
- To calculate speed using the **distance–time formula**.
- To display the speed on a **16×2 LCD display**.
- To alert overspeeding using a **buzzer**.
- To develop a simple and cost-effective traffic monitoring system.

COMPONENTS USED

The system consists of the following components:

- **Arduino UNO** – acts as the main controller
- **IR Sensors (2)** – used for vehicle detection
- **16×2 LCD Display (I2C)** – displays speed and messages
- **Buzzer** – indicates overspeeding
- **Breadboard** – for circuit connections
- **Jumper Wires** – for wiring
- **Power Supply (USB / Power Bank)**

SYSTEM ARCHITECTURE

The system architecture shows how different components interact with each other.

Flow:

- Vehicle → IR Sensor 1 → Arduino (Start Timer) → IR Sensor 2 → Arduino (Stop Timer) → Speed Calculation → LCD Display → Buzzer Alert
- The Arduino acts as the central unit that processes all inputs and generates outputs.

WORKING PRINCIPLE

- Two IR sensors are placed **1 meter apart**.
- When a vehicle crosses the first sensor, the Arduino starts a timer.
- When the vehicle crosses the second sensor, the timer stops.
- The time taken is calculated using the formula: **Speed = Distance / Time**
- The speed is calculated in m/s and converted to km/h.
- The result is displayed on the LCD.
- If speed exceeds 50 km/h, the buzzer is activated.

CODE

iot_pbl.ino

```cpp
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

const int sensor1 = A0;
const int sensor2 = A1;
const int buzzer  = 13;

unsigned long startTime = 0;
float distance = 1.0;   // meters between sensors

int firstSensor = 0;     // 0 = none, 1 = sensor1, 2 = sensor2

void setup() {
  pinMode(sensor1, INPUT);
  pinMode(sensor2, INPUT);
  pinMode(buzzer, OUTPUT);

  lcd.init();
  lcd.backlight();

  lcd.setCursor(0, 0);
  lcd.print("Speed Detector");
  delay(2000);
  lcd.clear();
}

void loop() {

  // Detect first sensor
  if (firstSensor == 0) {
    if (digitalRead(sensor1) == LOW) {
```

iot_pbl.ino

```cpp
  // Detect first sensor
  if (firstSensor == 0) {
    if (digitalRead(sensor1) == LOW) {
      startTime = millis();
      firstSensor = 1;
    }
    else if (digitalRead(sensor2) == LOW) {
      startTime = millis();
      firstSensor = 2;
    }
  }

  // If first was sensor1, wait for sensor2
  else if (firstSensor == 1) {

    if (digitalRead(sensor2) == LOW) {
      calculateSpeed();
    }

    // Timeout 3 seconds
    else if (millis() - startTime > 3000) {
      showNoVehicle();
    }
  }

  // If first was sensor2, wait for sensor1
  else if (firstSensor == 2) {

    if (digitalRead(sensor1) == LOW) {
      calculateSpeed();
    }

    // Timeout 3 seconds
```

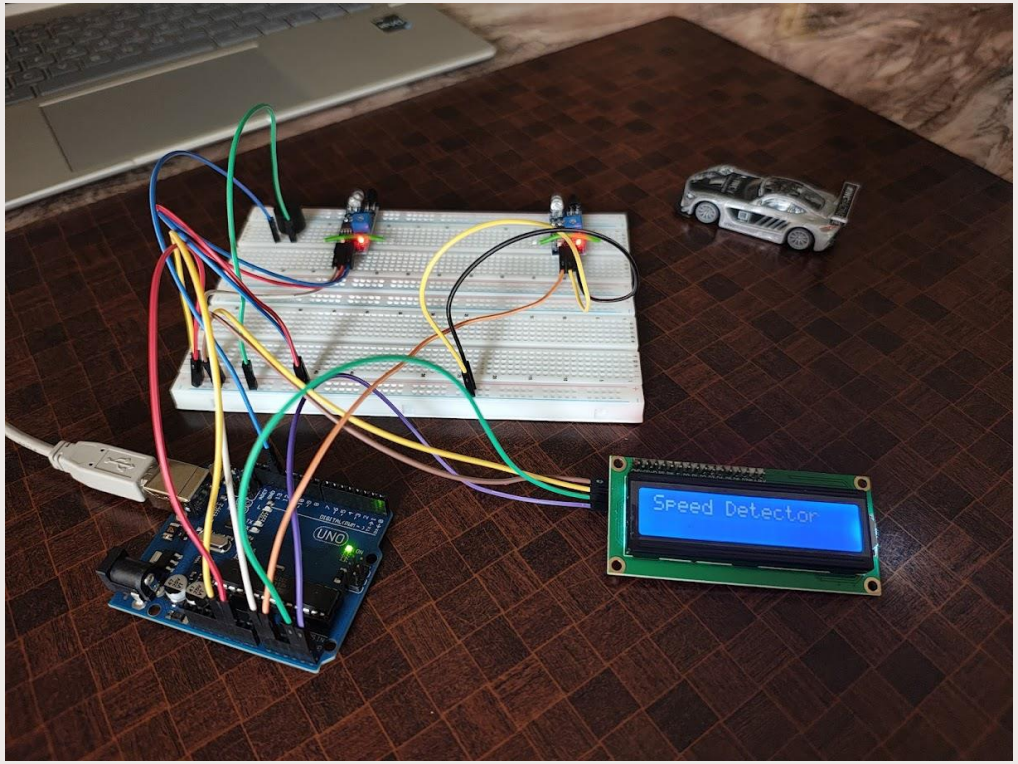
iot_pbl.ino

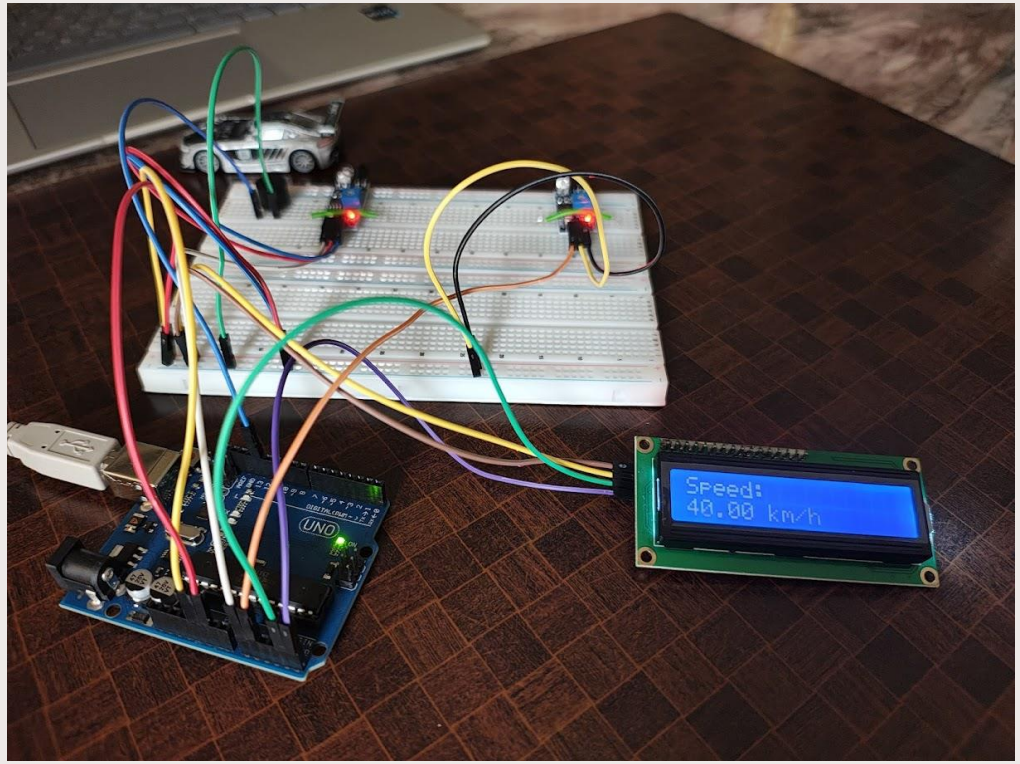
```
    // Timeout 3 seconds
    else if (millis() - startTime > 3000) {
      showNoVehicle();
    }
  }
}

void calculateSpeed() {

  unsigned long endTime = millis();
  unsigned long timeTaken = endTime - startTime;

  if (timeTaken > 0) {

    float speed = distance / (timeTaken / 1000.0);
    float speedKmph = speed * 3.6;

    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Speed:");
    lcd.setCursor(0, 1);
    lcd.print(speedKmph);
    lcd.print(" km/h");

    if (speedKmph > 50) {
      digitalWrite(buzzer, HIGH);
      delay(5000);   // Buzzer ON for 5 sec
      digitalWrite(buzzer, LOW);
    }
    else {
      delay(3000);
    }
```

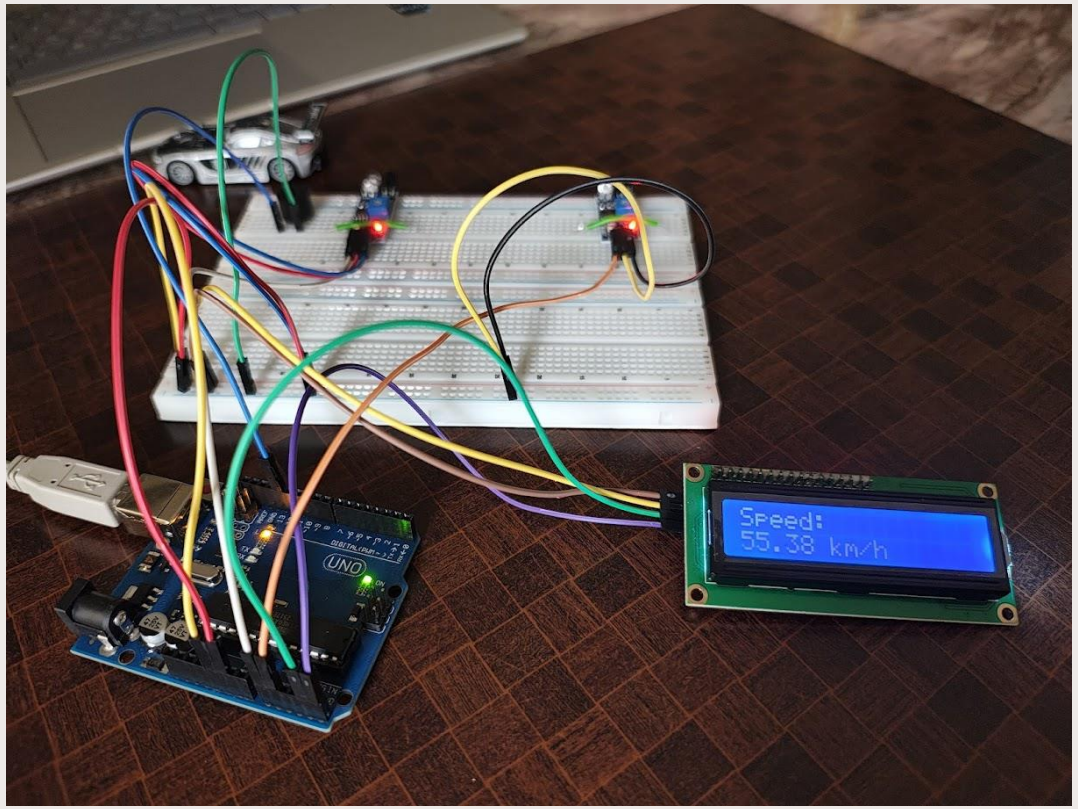
iot_pbl.ino

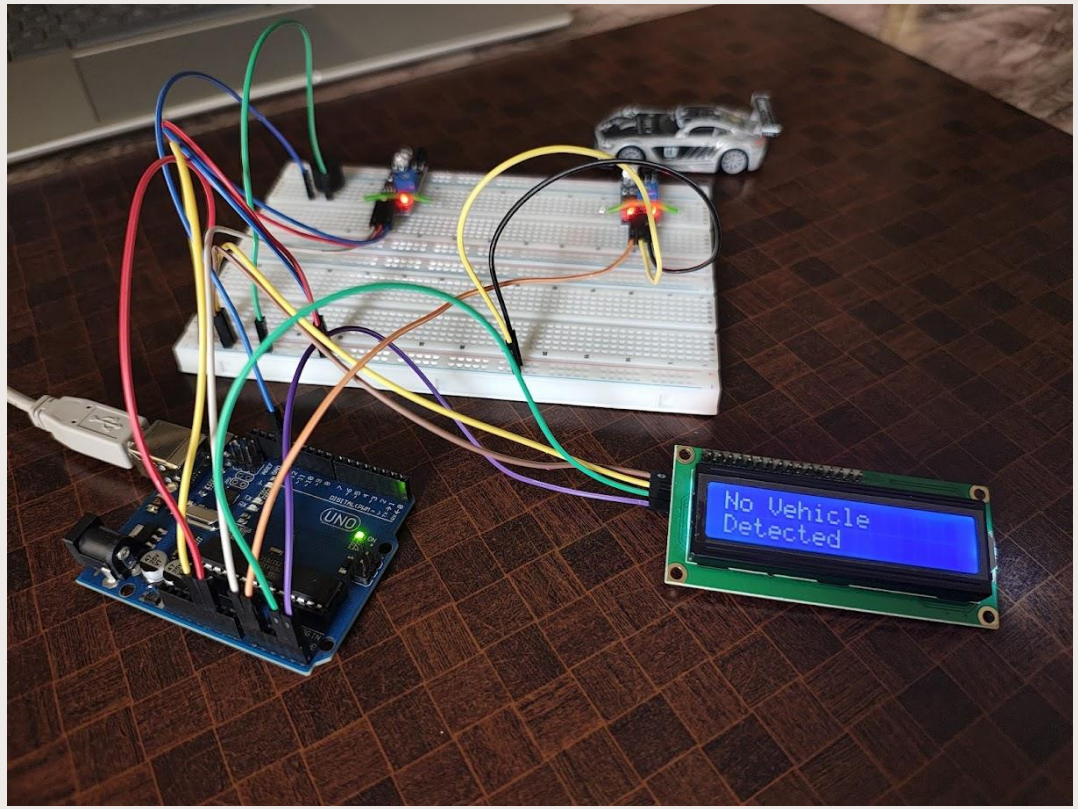
```
    lcd.setCursor(0, 0);
    lcd.print("Speed:");
    lcd.setCursor(0, 1);
    lcd.print(speedKmph);
    lcd.print(" km/h");

    if (speedKmph > 50) {
      digitalWrite(buzzer, HIGH);
      delay(5000);   // Buzzer ON for 5 sec
      digitalWrite(buzzer, LOW);
    }
    else {
      delay(3000);
    }

    lcd.clear();
  }

  firstSensor = 0;   // Reset
}

void showNoVehicle() {

  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("No Vehicle");
  lcd.setCursor(0, 1);
  lcd.print("Detected");
  delay(2000);
  lcd.clear();

  firstSensor = 0;   // Reset
}
```

RESULTS






Speed:
55.38 km/h


No Vehicle
Detected

ADVANTAGES AND LIMITATIONS

Advantages:

- Low-cost system
- Easy to build and understand
- Real-time speed measurement
- Portable and compact

Limitations:

- Accuracy depends on sensor placement
- Affected by ambient light conditions
- Not suitable for long-distance/highway use
- IR sensors have limited range

CONCLUSION

- The project successfully demonstrates a simple and effective method to measure vehicle speed using Arduino and IR sensors. It shows how basic electronic components can be used to build a real-time speed monitoring system.
- The system is reliable for small-scale applications and provides a clear understanding of embedded systems and sensor-based measurements.

THANK YOU